

Documento Técnico: Integración de APIs Externas Taller N°2

Felipe Acevedo y Antonio Arenas
Carrera de Ingeniería Civil Informática
Universidad Autónoma de Chile

18 de junio de 2026

Resumen

Este informe técnico detalla la arquitectura, el diseño y la implementación correspondientes a la integración de APIS al proyecto de software del semestre **uTrack**. El objetivo principal de la actividad radica en dotar a una aplicación móvil de gestión temporal y calendarios académicos de un contexto climático y diario de información en tiempo real, utilizando para ello herramientas de Flutter y Dart.

Índice

1. Introducción y Contexto	4
2. Objetivos del Taller y Solución Propuesta	4
2.1. APIs Seleccionadas y Justificación	4
3. Arquitectura de Integración y Flujo de Datos	4
3.1. Ciclo de Consulta	5
4. Detalles de Implementación Técnica	5
4.1. Consumo REST y Procesamiento JSON	5
4.2. Manejo de Errores y Robustez del Sistema	5
5. Diseño de Interfaz de Usuario (UI) Dinámica	6
6. Conclusión	6

1. Introducción y Contexto

El presente documento expone las actividades de diseño e ingeniería de software desarrolladas para dar cumplimiento a los requerimientos del Taller N°2. La temática del proyecto semestral, denominada **uTrack**, consiste en una plataforma móvil orientada al seguimiento, ordenamiento y priorización de la carga académica y tareas diarias de estudiantes universitarios.

Para la planificación diaria se tuvo en cuenta que cobra mayor valor cuando está provista de un contexto situacional externo. Por lo que se decidió integrar dos APIs de carácter público, con el fin de unificar en una sola vista la telemetría climática local y el calendario de Festividades y/o Feriados de la region.

2. Objetivos del Taller y Solución Propuesta

El núcleo de la solución radica en enriquecer el flujo de datos que el usuario visualiza al interactuar con el calendario semanal. Al seleccionar o deslizar un día específico, el sistema genera consultas paralelas y asíncronas para desplegar información del entorno de manera dinámica.

2.1. APIs Seleccionadas y Justificación

- **Open-Meteo API (Servicio Meteorológico):** Seleccionada debido a su alta precisión técnica . Para este proyecto, se parametrizó utilizando las coordenadas geográficas de la comuna de **Talca (Latitud: $-35,4264$, Longitud: $-71,6554$)**. Su integración permite prever variables climáticas que pudiesen alterar los traslados o la asistencia a evaluaciones presenciales, al ser local la App de momento para demostrar características se enfoca en talca pero se puede activar geolocalización para que cambie por zona también.
 - **Feriados de Chile API :** Servicio que indexa de forma oficial las festividades civiles y religiosas del país. Es fundamental para alertar automáticamente al estudiante si una jornada seleccionada corresponde a un día de cese de actividades académicas.
-

3. Arquitectura de Integración y Flujo de Datos

La integración de los servicios externos se diseñó bajo un esquema desacoplado y orientado a eventos concurrentes en el ciclo de vida del componente `StatefulWidget` en Flutter.

3.1. Ciclo de Consulta

1. **Inicialización:** El flujo de consulta HTTP se gatilla por primera vez en el método nativo `initState` para el día en curso, y de manera subsecuente mediante el método controlador `actualizarDia(int index)` cada vez que el usuario desliza horizontalmente el componente `PageView` adaptandose al día especificado de la semana.
 2. **Normalización Fecha:** El objeto `DateTime` de Dart se transforma a una cadena de texto estandarizada (`YYYY-MM-DD`) empleando extensiones de formateo de caracteres (`padLeft`).
 3. **Concurrencia de Red:** Se despachan solicitudes de tipo `GET` mediante el paquete oficial `http`.
-

4. Detalles de Implementación Técnica

4.1. Consumo REST y Procesamiento JSON

Los datos crudos transmitidos por los servidores externos en formato binario son decodificados mediante la librería `dart:convert` utilizando el método `jsonDecode`.

Para el microservicio meteorológico, la API retorna una colección indexada con la temperatura máxima esperada y un identificador numérico internacional estándar (WMO Weather Interpretation Code). Se estructuró un bloque de lógica condicional polimórfica para mapear dichos códigos numéricos a variables semánticas legibles en la interfaz (ej. `Código ≥ 95` → `"Tormenta"`) y asignar de forma dinámica un identificador gráfico de la colección de `Material Design` (`IconData`).

Para el microservicio de festividades, se procesa un arreglo dinámico aplicando el operador lineal `firstWhere`, comparando el campo clave `'fecha'` con el día en visualización activa del calendario.

4.2. Manejo de Errores y Robustez del Sistema

Para garantizar la continuidad operativa de la aplicación frente a problemas de conectividad o fallas en los servidores de origen, se implementaron tres mecanismos de control:

- **Bloques Try-Catch:** Toda operación de entrada/salida (I/O) de red se encapsula de manera estricta. Ante caídas de señal, problemas de resolución DNS o *timeouts*, el sistema captura la excepción de forma silenciosa, evitando el cierre forzado de la aplicación (*crash*) y alternando a un estado controlado de `"Sin conexión"`.
 - **Validación de Códigos de Estado (HTTP Status Codes):** Se evalúa explícitamente que sea estado de éxito `200 OK` antes de proceder con la acción a ejecutar.
-

5. Diseño de Interfaz de Usuario (UI) Dinámica

La información extraída de los endpoints remotos se unifica visualmente en un contenedor horizontal (`Row`) con alineación simétrica. Este bloque se insertó de manera estratégica inmediatamente debajo del texto cronológico principal y separado por divisores estéticos (`Divider`), conviviendo de forma limpia y sin alterar las listas de almacenamiento interno que renderiza el hijo `ListaContenidoDiario`.

- **Estado de Espera (Loading):** Mientras la latencia de red procesa la solicitud, la pantalla muestra de forma no invasiva la cadena `^actualizando...`.
- **Pintado Condicional :** Se muestra solo si es feriado mediante la verificación con un booleano de manera que solo si es información prioritaria se le mostrara al alumno, la info de clima se muestra siempre porque es algo que diariamente servira.

—

6. Conclusión

Como bien se esperaba la conexión de las APIs suma un aporte ajustado a las necesidades de la app tanto como estudiante o profesor sin cambiar en gran medida la interfaz para que siga manteniendo el orden lógico que se mantenía, la idea es lograr un aporte con las 2 APIs agregadas y se mantienen como se esperaba.